UNIT III

Game Theory & Job Sequencing

Handwritten study notes | B.Tech | Operations Research

Syllabus topics covered in this unit:

- Two-person zero-sum games: maximin-minimax principle, saddle point
- Games without saddle points: mixed strategies
- Dominance, arithmetic (oddments), algebraic and graphical methods
- Job sequencing: flow shop and job shop scheduling
- n jobs through 2 machines, n jobs through m machines, graphical method

1. Introduction to Game Theory

Game theory *is a mathematical study of decision making in situations of* ***conflict or competition****, where the outcome for one decision-maker depends on the decisions of the others (rivals). It was developed by* ***John von Neumann and Oskar Morgenstern*** *(1944, *Theory of Games and Economic Behaviour*).*

Important terms

- *Player: each competitor/decision maker (A, B). Two-person game = two players*
- *Strategy: complete plan of action for a player.* ***Pure strategy*** *- one fixed course always chosen;* ***mixed strategy*** *- a probability mix of pure strategies*
- *Pay-off matrix: table of gains/losses for each pair of strategies. By convention entries show payoff to the* ***row player A (maximiser)****; B (minimiser) pays*
- *Value of the game (V): expected payoff to A when both play optimally.* ***Fair game: V = 0.*** *Strictly determinable game: maximin = minimax*
- *Zero-sum game: gain of one player = loss of the other (sum of payoffs = 0)*
- *Optimal strategy: the strategy that gives a player the most favourable outcome regardless of what the opponent does*

Assumptions of a two-person zero-sum game

- *Each player has a finite number of strategies, known to both. Players act rationally and simultaneously*
- *Each player tries to maximise the minimum gain (A) or minimise the maximum loss (B); payoffs are known with certainty*

Types of games

Basis	Types
Number of players	Two-person, n-person
Sum of payoffs	Zero-sum, non-zero-sum (variable-sum)
Number of strategies	Finite, infinite
Strategy used	Pure-strategy, mixed-strategy
Co-operation	Co-operative, non-co-operative

2. Maximin - Minimax Principle and Saddle Point

- Player A (maximiser) finds the **minimum of each row**, then takes the **maximum** of those: maximin value $\underline{V} = \max_i \min_j a_{ij}$
- Player B (minimiser) finds the **maximum of each column**, then takes the **minimum** of those: minimax value $\overline{V} = \min_j \max_i a_{ij}$
- Always maximin ≤ minimax. If **maximin = minimax = V** the game has a saddle point: the cell is the **minimum of its row and maximum of its column**. Pure strategies are optimal and V is the value of the game

Example 1 - game with saddle point

A \ B	B1	B2	B3	Row min
A1	1	3	1	1
A2	0	-4	-3	-4
A3	1	5	-1	-1
Col max	1	5	1	

Maximin = max(1, -4, -1) = 1. Minimax = min(1, 5, 1) = 1. Equal -> saddle point exists. Saddle points: (A1, B1) and (A1, B3). **Optimal: A plays A1, B plays B1 or B3, Value V = 1** (game favours A).

Note: a game may have more than one saddle point; all have the same value. If V = 0 it is a **fair** game; if no saddle point exists we use **mixed strategies**.

3. Games without Saddle Point - Mixed Strategies

When maximin ≠ minimax, neither player can fix one pure strategy (the opponent could exploit it). Each player chooses strategies **at random with fixed probabilities**, so that the expected payoff is optimal. A plays strategies with probabilities $p_1, p_2, \ldots$ ($\sum p_i = 1$); B with $q_1, q_2, \ldots$ ($\sum q_j = 1$).

Minimax theorem (von Neumann): for every finite two-person zero-sum game there exists a value V and optimal (pure or mixed) strategies for both players such that *max-min of expected payoff = min-max of expected payoff = V*.

Formulae for a 2 × 2 game

Pay-off matrix [a b ; c d] (A's rows 1, 2; B's columns 1, 2) with no saddle point:

$p_1 = (d - c) / (a + d - b - c)$, $p_2 = 1 - p_1$

$q_1 = (d - b) / (a + d - b - c)$, $q_2 = 1 - q_1$

Value $V = (ad - bc) / (a + d - b - c)$

Example 2

A \ B	B1	B2	Row min
A1	2	4	2
A2	6	3	3
Col max	6	4	

Maximin = 3, minimax = 4 -> no saddle point. Here a=2, b=4, c=6, d=3, a+d-b-c = -5.

$p_1 = (3 - 6)/(-5) = 3/5$, $p_2 = 2/5$. $q_1 = (3 - 4)/(-5) = 1/5$, $q_2 = 4/5$.

$V = (2\cdot3 - 4\cdot6)/(-5) = (-18)/(-5) =$ **18/5 = 3.6**

Check: A's expected payoff vs B1 = 2(3/5)+6(2/5) = 3.6; vs B2 = 4(3/5)+3(2/5) = 3.6 (equal, hence optimal).

Arithmetic (oddments) method for 2 × 2

- Subtract the entries of **column 2 from column 1** (ignore sign) and write the result against the rows in reverse: these are the row **oddments**. Repeat with the rows for the column oddments
- Probability = oddment / sum of oddments

	B1	B2	Row oddment	Prob.
A1	2	4	|6-3| = 3	$p_1 = 3/5$
A2	6	3	|2-4| = 2	$p_2 = 2/5$
Col oddment	|4-3| = 1	|2-6| = 4		
Prob.	$q_1 = 1/5$	$q_2 = 4/5$		

Row oddments: A1 gets |c-d| = 3, A2 gets |a-b| = 2 (the difference of the OTHER row, taken

across). Column oddments: B1 gets |b-d| = 1, B2 gets |a-c| = 4. Same answer as the formula; V = (2·3 + 6·2)/5 = 3.6.

Algebraic method (equal expected payoffs)

Let A play A1 with probability p and A2 with (1-p). Against B1 and B2, expected payoffs must be equal: 2p + 6(1-p) = 4p + 3(1-p) => 6 - 4p = 3 + p => p = 3/5. Substitute to get V = 2(3/5)+6(2/5) = 3.6. In the same way B's q is found from 2q + 4(1-q) = 6q + 3(1-q) => q = 1/5.

4. Rules of Dominance

Dominance reduces the size of a payoff matrix by deleting strategies that will never be used by a rational player.

- *Row rule: if every element of row r is ≥ the corresponding element of row s, then row s (smaller) is dominated -> delete row s (A never plays it)*
- *Column rule: if every element of column r is ≤ the corresponding element of column s, then column s (larger) is dominated -> delete column s (B never plays it)*
- *Modified dominance: a row (column) may also be dominated by the **average / convex combination** of two or more other rows (columns)*
- *Dominance can be applied repeatedly. The probabilities of deleted strategies are zero*

Example 3 - solve using dominance

A \ B	B1	B2	B3
A1	1	7	2
A2	6	2	7
A3	5	1	6

- *A3 = (5, 1, 6) ≤ A2 = (6, 2, 7) in every column, so A2 dominates A3 -> delete A3*
- *Now B3 = (2, 7) ≥ B1 = (1, 6) in each row; B minimises, so B1 dominates B3 -> delete B3*

A \ B	B1	B2
A1	1	7
A2	6	2

2 × 2 game with a=1, b=7, c=6, d=2 (no saddle: maximin 2, minimax 6). a+d-b-c = -10.

$p_1 = (2 - 6)/(-10) = 2/5$, $p_2 = 3/5$; $q_1 = (2 - 7)/(-10) = 1/2$, $q_2 = 1/2$; $V = (2 - 42)/(-10) = 4$

Optimal: *A plays (A1, A2, A3) with (2/5, 3/5, 0); B plays (B1, B2, B3) with (1/2, 1/2, 0); Value = 4.*

5. Graphical Method for 2 × n or m × 2 Games

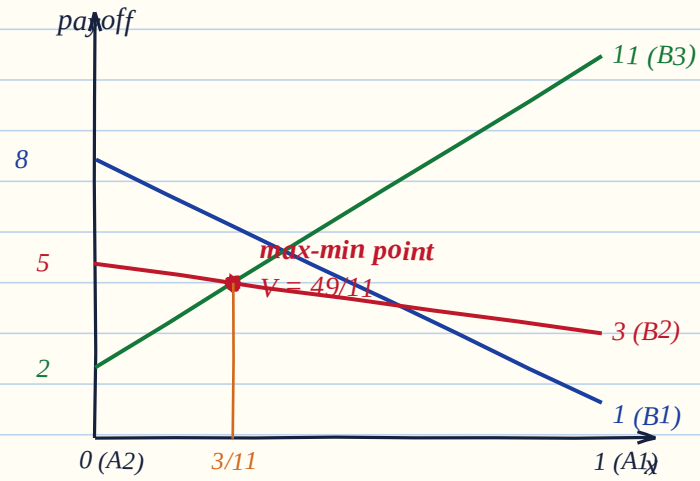
*Used when one player has only **two** strategies. Plot the expected payoff against each opposing pure strategy as a straight line; find the **highest point of the lower envelope** (for A) or the **lowest point of the upper envelope** (for B).*

- *Let A play A1 with probability x and A2 with 1 - x ($0 \le x \le 1$)*
- *For each column j of B draw $E_j(x) = a_{1j}x + a_{2j}(1 - x)$. A wants to **maximise the minimum** of these lines*
- *The point where the lower envelope is **highest** gives x* and V. The two lines meeting there identify B's active strategies; solve the 2 × 2 sub-game for q*

Example 4: 2 × 3 game

A \ B	B1	B2	B3
A1	1	3	11
A2	8	5	2

$E_1 = 8 - 7x$, $E_2 = 5 - 2x$, $E_3 = 2 + 9x$.



***Lower envelope** peaks where E_2 meets E_3 (B1 never used):*

$2 + 9x = 5 - 2x \Rightarrow x = 3/11$

$V = 2 + 27/11 = 49/11 \approx 4.45$

Sub-game B2, B3 gives B's mix:

$3q + 11(1-q) = 5q + 2(1-q)$

$\Rightarrow q = 9/11$ (B2), $2/11$ (B3)

***Solution:** A plays A1 with 3/11, A2 with 8/11. B plays B1 with 0, B2 with 9/11, B3 with 2/11. Value of game V = 49/11.*

Check A's rows: A1 -> 3(9/11) + 11(2/11) = 49/11; A2 -> 5(9/11) + 2(2/11) = 49/11 (equal, so optimal).

Other methods (names only)

- *Linear programming method: any m × n game can be converted to an LP; A's problem is the dual of B's problem (see Unit I duality)*
- *Matrix (inverse) method for square games; iteration/Brown's method for approximate solutions*

Linear programming formulation of a game (idea)

Make all payoffs positive (add a constant if needed). A plays x_i ($\sum x_i = 1$), V = value of game.

Player A: Max V s.t. $\sum_i a_{ij} x_i \geq V$ (for every column j), $\sum x_i = 1$, $x_i \geq 0$.

Put $y_i = x_i/V$: Min $Z = \sum y_i$ s.t. $\sum a_{ij} y_i \geq 1$, $y \geq 0$ and then $V = 1/Z$. B's problem is the DUAL of A's problem.

Applications and limitations of game theory

- Uses: pricing and advertising competition between firms, bidding and auctions, military strategy, labour-management bargaining, tender decisions
- Limits: assumes rival is rational and both know all payoffs; very large games are hard; real life seldom has exactly two players with strictly opposed interests

6. Job Sequencing - Introduction

Sequencing is the problem of finding the **order in which n jobs should be processed on m machines** so that a chosen measure (total elapsed time, idle time, cost) is optimised. Total possible sequences = $(n!)^m$ in general.

Terminology

- Processing time (t_{ij}): time job i needs on machine j (independent of the order)
- Processing order: the order of machines through which a job must pass (e.g. A -> B -> C)
- Total elapsed time (T): time from start of the first job on the first machine to completion of the last job on the last machine
- Idle time: time a machine is waiting with no job to process, within [0, T]
- No-passing rule: the same order of jobs is kept on every machine

Assumptions

- Machines are always available; one machine does one job at a time; a job is on only one machine at a time
- Once begun, a job is processed to completion (no pre-emption); transit time is negligible (included in t_{ij})

Types of scheduling problems

Type	Meaning
Flow shop	all jobs have the SAME machine order (assembly line, e.g. M1 -> M2)
Job shop	different jobs have DIFFERENT routings; more complex
n jobs, 2 machines	Johnson's rule - exact optimum
n jobs, 3 machines	Johnson's rule if conditions satisfied
n jobs, m machines	extension of Johnson's rule (special conditions)
2 jobs, m machines	graphical method

7. Processing n Jobs through Two Machines (Johnson's Rule)

Johnson's algorithm (minimises total elapsed time for jobs going M1 -> M2):

1. *List the processing times of all jobs on M1 and M2.*
2. *Find the **smallest** time in the whole list.*
3. *If it is on **M1**, place that job at the **first** free position of the sequence; if it is on **M2**, place it at the **last** free position.*
4. *Delete that job; repeat steps 2-3 for the remaining jobs. Ties: pick either (several optimal sequences possible).*

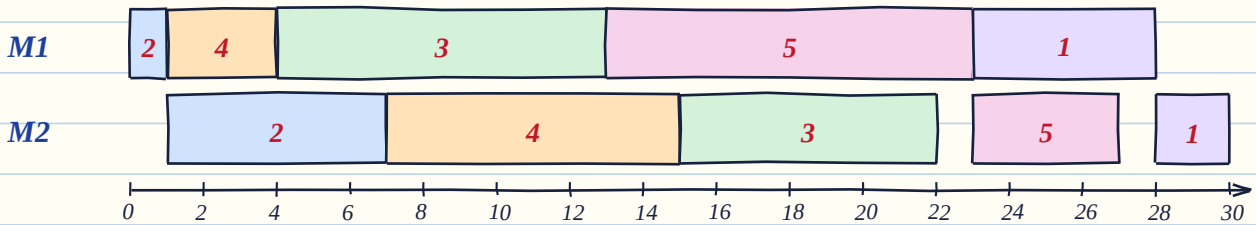
Example 5: five jobs, two machines (hours)

Job	1	2	3	4	5
M1	5	1	9	3	10
M2	2	6	7	8	4

- Smallest = 1 (Job 2 on M1) -> first position: 2 _ _ _ _
- Next smallest = 2 (Job 1 on M2) -> last position: 2 _ _ _ 1
- Next = 3 (Job 4 on M1) -> next first: 2 4 _ _ 1. Next = 4 (Job 5 on M2) -> last free: 2 4 _ 5 1. Remaining Job 3 -> middle

Optimal sequence: 2 - 4 - 3 - 5 - 1

Job	M1 in	M1 out	M2 in	M2 out
2	0	1	1	7
4	1	4	7	15
3	4	13	15	22
5	13	23	23	27
1	23	28	28	30



Total elapsed time = 30 hours.

Idle time on M1 = 30 - (5+1+9+3+10) = 2 hrs (Job 1 leaves M1 at 28). Idle time on M2 = 3 hrs = 1 (initial wait) + 1 (between Jobs 3 and 5) + 1 (between Jobs 5 and 1).

Formula: Idle time on M2 = ∑ (start of job on M2 - finish of previous job on M2) + initial wait (= $t_{1,M1}$); Idle time on M1 = T - ∑ $t_{i,M1}$.

8. Processing n Jobs through Three Machines

*Johnson's rule extends to 3 machines **only if** at least one of the following conditions holds:*

> *(i) Minimum time on M1 ≥ Maximum time on M2 or (ii) Minimum time on M3 ≥ Maximum time on M2*
> *Then replace the 3 machines by two **fictitious machines**: G = M1 + M2 and H = M2 + M3, and apply Johnson's rule to (G, H).*

Example 6: seven jobs, machines A, B, C

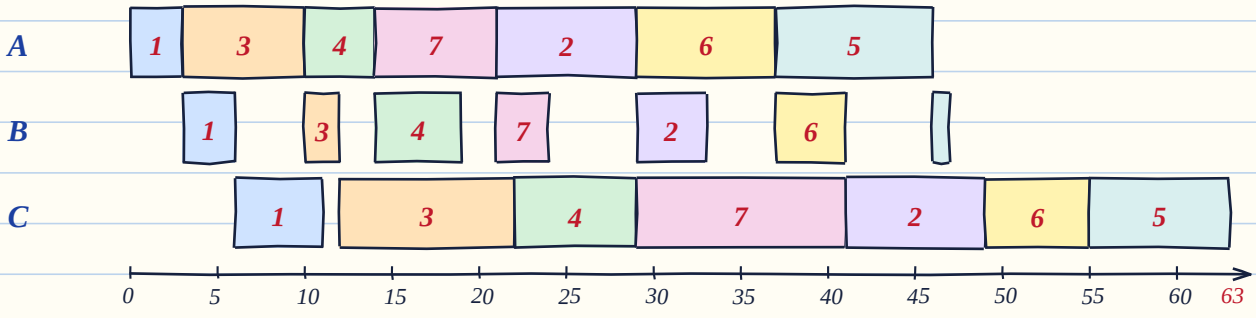
Job	1	2	3	4	5	6	7
A	3	8	7	4	9	8	7
B	3	4	2	5	1	4	3
C	5	8	10	7	8	6	12

Min A = 3, Max B = 5, Min C = 5. Condition (ii): min C = 5 ≥ max B = 5 is satisfied. Fictitious machines:

Job	1	2	3	4	5	6	7
G=A+B	6	12	9	9	10	12	10
H=B+C	8	12	12	12	9	10	15

*Johnson's rule on (G, H) gives the optimal sequence **1 - 3 - 4 - 7 - 2 - 6 - 5**. Now find the in-out times on the **original** machines A, B, C:*

Job	A in-out	B in-out	C in-out
1	0 - 3	3 - 6	6 - 11
3	3 - 10	10 - 12	12 - 22
4	10 - 14	14 - 19	22 - 29
7	14 - 21	21 - 24	29 - 41
2	21 - 29	29 - 33	41 - 49
6	29 - 37	37 - 41	49 - 55
5	37 - 46	46 - 47	55 - 63



Minimum total elapsed time = 63 hours.
Idle time: machine A = 17 hrs, machine B = 41 hrs, machine C = 7 hrs (T - total processing time on that machine).

9. Processing n Jobs through m Machines

*For m > 3 machines the same idea is used. If **either** condition below holds, replace the m machines by two fictitious machines G and H and use Johnson's rule.*

Condition: $\min t_{i1} \geq \max t_{ij}$ *or* $\min t_{im} \geq \max t_{ij}$ *for* $j = 2, 3, \ldots, m-1$ *(all middle machines)*
$G_i = t_{i1} + t_{i2} + \ldots + t_{i,m-1}$ $H_i = t_{i2} + t_{i3} + \ldots + t_{im}$

Example 7: four jobs, four machines

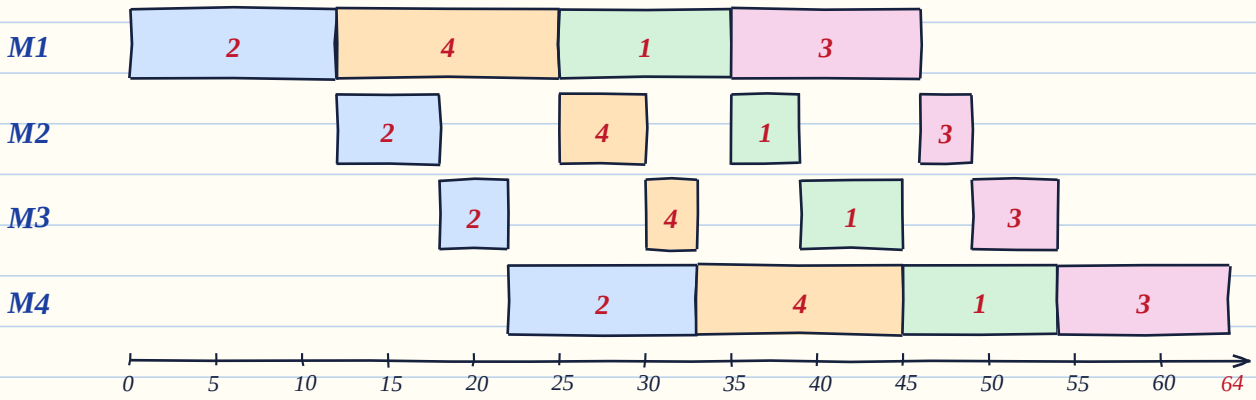
Job	1	2	3	4
M1	10	12	11	13
M2	4	6	3	5
M3	6	4	5	3
M4	9	11	10	12

Min M1 = 10 ≥ Max(M2) = 6 and Max(M3) = 6, so the condition holds. G = M1+M2+M3, H = M2+M3+M4.

Job	1	2	3	4
G	20	22	19	21
H	19	21	18	20

Smallest value = 18 (Job 3 on H) -> last; next 19 (Job 1 on H) -> second last; 20 (Job 4 on H) -> next; then Job 2. Sequence = ***2 - 4 - 1 - 3****.*

Job	M1	M2	M3	M4
2	0-12	12-18	18-22	22-33
4	12-25	25-30	30-33	33-45
1	25-35	35-39	39-45	45-54
3	35-46	46-49	49-54	54-64



Total elapsed time = 64 hrs. *Idle times: M1 = 18, M2 = 46, M3 = 46, M4 = 22 hrs.*

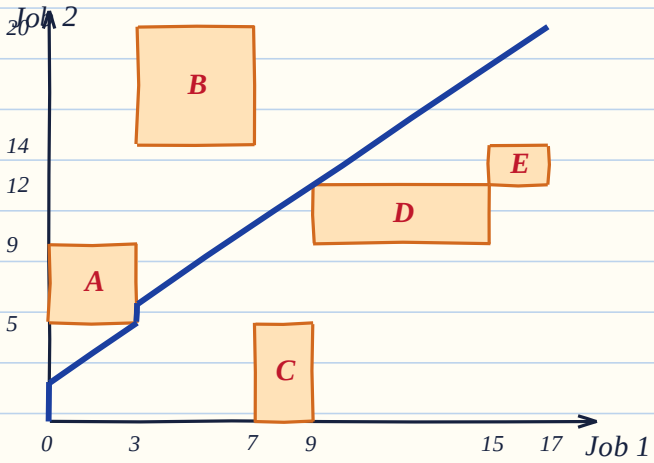
10. Two Jobs through m Machines - Graphical Method

Two jobs, each to be processed on the ***same m machines but in different orders****. A graph is used to find the minimum elapsed time.*

- *Mark Job 1's processing times along the* ***x-axis*** *and Job 2's along the* ***y-axis****, in their respective machine orders*
- *For each machine draw the* ***rectangle (block)*** *formed where both jobs need that machine - the two jobs cannot use it together, so the path cannot pass through the block*
- *From origin O to the end point draw a path using only horizontal (Job 1 works, Job 2 waits), vertical (Job 2 works) and 45° diagonal (both work) segments, avoiding blocks*
- *Elapsed time = horizontal length + vertical length + length of the diagonal pieces measured along one axis. Choose the path with the* ***fewest non-diagonal moves*** *(diagonals save time)*

Example 8

Job 1 sequence	A	B	C	D	E
Time	3	4	2	6	2
Job 2 sequence	C	A	D	E	B
Time	5	4	3	2	6



Optimal path (blue):

O -> (0,2): vertical (Job 2 alone, 2 hrs)
-> (3,5): diagonal (both work, 3 hrs)
-> (3,6): vertical (1 hr)
-> (17,20): diagonal (14 hrs)

Elapsed time = 2+3+1+14 = 20 hrs

= total time of Job 2 (20),
Job 1 waits 2 + 1 = 3 hrs.

Result: *minimum total elapsed time = 20 hours. Idle time for Job 1 = 20 - 17 = 3 hrs; for Job 2 = 0.*

Reading the graph: a diagonal step cannot enter a shaded block because both jobs would need the same machine at the same time.

11. Job Shop Scheduling & Dispatching Rules

*In a **job shop** every job may need a different set of machines in a different order, and many jobs compete for the same machine. Exact optimal solutions are too hard (up to $(n!)^m$ schedules), so shops use **priority (dispatching) rules** to choose which waiting job a free machine should process next.*

Rule	Meaning	Effect
FCFS	First come, first served	Fair; poor flow time
SPT	Shortest processing time first	Minimises mean flow time
LPT	Longest processing time first	Good machine balance
EDD	Earliest due date first	Reduces maximum lateness
Slack	Due date - now - remaining work (least first)	Protects urgent jobs
CR	Critical ratio = time left to due date / work left	CR < 1 means job is late

Performance measures

- *Makespan = completion time of the last job (total elapsed time); Flow time F_i = completion time - arrival time; Lateness L_i = completion - due date; Tardiness = max(0, L_i)*
- *Other measures: machine utilisation, work-in-process inventory, number of tardy jobs*

Example - single machine, 4 jobs (processing times 4, 2, 7, 3 days)

Rule	Order	Completion times	Mean flow time
FCFS	J1 J2 J3 J4	4, 6, 13, 16	39/4 = 9.75
SPT	J2 J4 J1 J3	2, 5, 9, 16	32/4 = 8.0

SPT gives the smaller mean flow time (8 days). This is the standard result: on one machine, **SPT minimises mean flow time**, and **EDD minimises maximum lateness**.

Flow shop vs Job shop

Point	Flow shop	Job shop
Routing	same order for all jobs	different for each job
Layout	product (assembly line)	process (functional)
Volume / variety	high volume, low variety	low volume, high variety
Number of schedules	n! (permutation)	$(n!)^m$
Technique	Johnson's rule etc.	dispatching rules, heuristics

12. Quick Revision & Important Questions

Game theory: maximin ≤ minimax; saddle point if equal; 2×2 formulae p_1 = (d-c)/(a+d-b-c), V = (ad-bc)/(a+d-b-c); dominance rows (delete smaller), columns (delete larger).

Sequencing: Johnson: smallest on M1 -> first, smallest on M2 -> last; 3 machines need min M1 ≥ max M2 or min M3 ≥ max M2; G = M1+M2, H = M2+M3; Idle time on last machine...

Likely questions: (1) Define a two-person zero-sum game, explain maximin-minimax principle. (2) Solve a 2×2 / 3×3 game with dominance. (3) Graphical solution of 2×n game.
(4) Johnson's rule: n jobs 2 machines with Gantt chart & idle time. (5) n jobs 3 machines. (6) 2 jobs m machines graphical method. (7) Differences between flow shop and job shop.